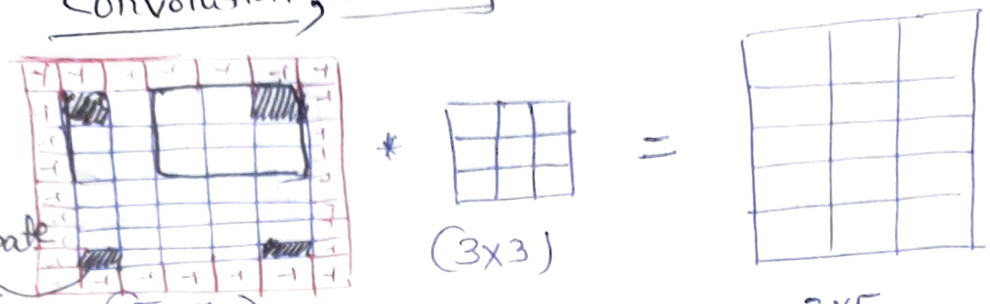
Cat-5 Lec-3
13 Oct 2024

Convolution, Padding, and Stride



Participate in just one convolution operation

(5x7) (mxn) * (3x3) (fxf) = 3x5 (Valid Convolution)

$(m-f+1) \times (n-f+1) = (5-3+1) \times (7-3+1) = 3 \times 5$

→ Disadvantage: Corner Pixels don't contribute as much in feature detection.

(5x7)
After Padding = 1
(7x9) mxn * (3x3) filter ⟹ 5x7 (Same Convolution)

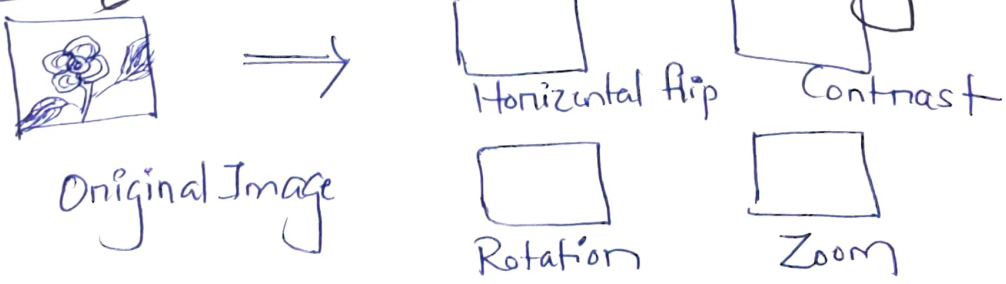
Valid Convolution → No Padding
Same Convolution → Pad such that Output is same as input.

→ Layers: Conv2D(16, 3, padding='same', activation='relu')

→ Stride: (1,1) — move in Y axis 1 pixel; move in X axis 1 pixel

stride = (2,2)

→ Data Augmentation to address Overfitting:



```
import matplotlib.pyplot as plt
import numpy as np
import cv2          ---- OpenCV
import os
import PIL          ---- Image Library in Python (Pillow Library)
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.models import Sequential
```

→ dataset_url = "https://storage.googleapis.com/download.tensorflow.org/example_images/flower_photos.tgz"

```
data_dir = tf.keras.utils.get_file('flower_photos',
                                   origin=dataset_url,
                                   cache_dir='.',        # current directory
                                   untar=True)           # unzip it
```

data_dir → '.\\datasets\flower_photos' → Path

→
```
import pathlib
data_dir = pathlib.Path(data_dir)
```

data_dir → WindowsPath('datasets/flower_photos') → Windows Path Object

```
list(data_dir.glob('*/*.jpg'))   # go through all folders, check .jpg in data_dir.
image_count = len(                ) - 3670
roses = list(data_dir.glob('roses/*'))
roses[:5]
```

→ PIL.Image.open(str(roses[0]))

→ tulips = list(data_dir.glob('tulips/*'))

PIL.Image.open(str(tulips[0]))

→
```
flowers_images_dict = {
    'roses': list(data_dir.glob('roses/*')),
    'daisy':
    'dandelion':
    'sunflower':
    'tulips':
}
```

'daisy/*'
'dandelion/*'
'sunflowers/*'
'tulips/*'

→ flowers_images_dict['daisy']
↳ all the file path of daisy images.

→ flowers_labels_dict = {
'roses' : 0,
'daisy' : 1,
'dandelion' : 2,
'sunflowers' : 3,
'tulips' : 4, }

} labels dictionary.

→ img = cv2.imread(path)
str(flowers_images_dict['roses'][0])

→ img → numpy array (3D)
→ img.shape → (240, 179, 3)

→ cv2.resize(img, (180, 180)).shape → (180, 180, 3)
making same dimension for all images.

X, y = [], []
for flower_name, images in flowers_images_dict.items():
print(flower_name)
print(len(image))

(180, 180, 3)
X Y RGB

roses
641
daisy 633
dandelion 898
sunflow — 699
tulips — 799

for image in images:
img = cv2.imread(str(image))
resized_img = cv2.resize(img, (150,150))
X.append(resized_img)
y.append(flowers_labels_dict[flower_name])

X[0] ——— 3D array (list)

↳ convert to numpy array

```
X = np.array(X)
y = np.array(y)
```

→ # Train Test Split

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=0, test_size=0.25)
```

len(X_train) —— 2752

len(X_test) —— 918

X[10]

↳ array([[[[1,16,7], [1,16,7], [1,17,9] ...]]]) (160, 160, 3)

X_train_scaled = X_train / 255 # Scaling

X_test_scaled = X_test / 255

Building Model

```
model = Sequential([
    layers.Conv2D(16, 3, padding='same', activation='relu'),   # 16 # of filters, of size 3x3
    layers.MaxPooling2D(),
    layers.Conv2D(32, 3, 'same', 'relu'),
    layers.MaxPooling2D(),
    layers.Conv2D(64, 3, 'same', 'relu'),
    layers.MaxPooling2D(),
    layers.Flatten(),                                          # 2D array
    layers.Dense(128, activation='relu'),
    layers.Dense(5)                                            # num_classes = 5 define
])
model.compile(optimizer='adam',
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=['accuracy'])
```

, activation = 'softmax'

model.fit(X_train_scaled, y_train, epochs = 30)

Training (loss : 0.0037 accuracy : 0.9993)

evaluate on Test data

model.evaluate(X_test_scaled, y_test)

[2.3827 , 0.6535]

65%

Overfitting

Training (99%) Testing = (65%)

predictions = model.predict (X_test_scaled)

predictions.

score = tf.nn.softmax(predictions[0])

score

np.argmax(np.array([0.78, 1.23, 5])) — 2

np.argmax(score)

(x1) — daisy from

y_test[0] → (1)

Data augmentation :- Tensorflow API

→ data_augmentation = keras.Sequential([

layers.experimental.preprocessing.RandomZoom(0.3),

RandomContrast(0.1)

Random Rotation(0.2)

])

plt.axis('off')

plt.imshow(X[0]) — ploting 1st Image from training samples.

→ data_augmentation(X)[0] → index

↳ Function

↳ tf.tensor : shape = (180, 180, 3) , dtype = uint8 , numpy =

array([[[ ],

[ ]

[[ ]]

[[ ]

[[ ]]],)

→ plt.axis('off')

plt.imshow(data_augmentation(X)[0].numpy().astype("uint8"))

↳ augmented randomzoom image.

```
data_augmentation = keras.Sequential( [
    layers.experimental.preprocessing.RandomFlip("horizontal",
                                                 input_shape=(img_height,
                                                              img_width,
                                                              3)),
    layers.experimental.preprocessing.RandomRotation(0.1),
    layers.experimental.preprocessing.RandomZoom(0.1),
] )
```

→
```
num_classes = 5
model = Sequential( [
    data_augmentation,
    layers.Conv2D(16, 3, padding='same', activation='relu'),
    layers.MaxPooling2D(),
    layers.Conv2D(32, 3, padding='same', activation='relu'),
    layers.MaxPooling2D(),
    layers.Conv2D(64, 3, padding='same', activation='relu'),
    layers.MaxPooling2D(),
    layers.Dropout(0.2),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(num_classes)
] )

model.compile(optimizer='adam',
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=['accuracy'])
model.fit(X_train_scaled, y_train, epochs=30)
```

↳ Epoch 30/30 — 86/86 — 1s 10ms/step — loss: 0.26 — accuracy: 0.90 ← train accuracy

→
```
model.evaluate(X_test_scaled, y_test)
```

↳ 29/29 — — — loss: 90 — accuracy: 0.75

[0.90, 0.75]

test loss — test accuracy